

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA09 - Programming in Java

FINAL ASSIGNMENT REPORT

Real-Time Railway Ticket Reservation System

Java OOP, Collections, Multithreading, AWT GUI, File Handling, Serialization and JDBC

Course Code & Name	CSA09 - Programming in Java
Slot	Slot A
Course Outcome (CO)	CO4 - Develop robust object-oriented, multithreaded and database-oriented Java applications by applying collections, synchronization, exception handling, GUI programming and file/database persistence.
Bloom's Taxonomy Level	Level 4 (L4) - Analyze
SDG Mapping	SDG 8 (Decent Work & Economic Growth), SDG 9 (Industry, Innovation & Infrastructure), SDG 16 (Peace, Justice & Strong Institutions)
Student Name	Chethrika Sri KM
Register Number	192511112
Programming Language	Java (JDK 8+ / Java SE)
Submission Date	September 2026

1. Problem Statement

A railway reservation system must manage passengers, trains, coaches, seats, tickets and payments while supporting concurrent booking requests. The main technical challenge is preventing two threads from reserving the same seat at the same time while keeping passenger validation, ticket creation, cancellation and persistence consistent.

The project therefore combines object-oriented modelling, Java Collections, custom exception handling, multithreading with synchronization, an AWT graphical interface, object serialization, and a supplied SQL database design. The implementation is intentionally kept in a single Java source file so that it is easy to compile and understand in a student development environment.

2. Objective

- Model railway entities using classes, an abstract Person base class and Passenger inheritance.
- Apply encapsulation, constructor overloading, method overriding and runtime polymorphism.
- Use ArrayList, HashSet, HashMap, Generics and Iterator for railway data management.
- Handle invalid passenger information, unavailable seats and booking/cancellation failures through custom exceptions.
- Demonstrate concurrent booking using Runnable threads, synchronized booking and thread priorities.
- Build a Java AWT GUI with controls, menus, layout managers and ActionListener event handling.
- Persist ticket objects using ObjectOutputStream/ObjectInputStream and provide SQL schema/CRUD operations for relational persistence.

3. Requirements and Environment Used

3.1 Software & Hardware Environment

Component	Specification / Tool
Operating System	Windows 11 / Windows 10 or equivalent
Programming Language	Java SE; code is compatible with standard javac execution
IDE	Visual Studio Code
GUI Toolkit	Java AWT (java.awt.*, java.awt.event.*)
Collections	ArrayList, HashMap, HashSet, Iterator
Persistence	Object Serialization; database/schema.sql and sample_operations.sql
Version Control	Git / GitHub

3.2 Java Concepts, Data Structures & Standard Libraries Used

Construct / Library	Role in the System
Classes & Objects	Passenger, Train, Coach, Seat, Ticket, Payment and service classes model railway information.
Abstract Class & Inheritance	Person is abstract; Passenger extends Person.
Overloading & Overriding	Train has overloaded constructors; Passenger and domain classes override toString/getRole where applicable.
HashMap	Maps train numbers, passenger IDs and ticket IDs for direct lookup.
HashSet	activeBookings stores unique train/seat booking keys and prevents duplicate active bookings.
ArrayList	Stores coaches, seats and ticket collections in ordered lists.
Generics & Iterator	Typed collections are used throughout; Iterator explicitly traverses trains.
Custom Exceptions	ReservationException, InvalidPassengerException, SeatUnavailableException and BookingException.
Multithreading & Runnable	BookingTask represents concurrent booking requests executed by multiple threads.
Synchronization	ReservationService.bookTicket() and cancelTicket() are synchronized to protect shared booking state.
AWT GUI & Listeners	Frame, Panel, Label, TextField, TextArea, Button, MenuBar and ActionListener create the desktop interface.
Object Serialization	FileManager saves and restores List<Ticket> through ObjectOutputStream/ObjectInputStream.
SQL/JDBC Preparation	The database folder supplies relational schema and CRUD SQL for later JDBC integration.

3.3 Time & Space Complexity Summary

Operation	Time	Space	Reason
HashMap lookup	O(1) average	O(1) auxiliary	Direct key lookup.
Seat search	O(S)	O(1) auxiliary	Sequential search through seats/coaches.
Iterator traversal	O(T)	O(1) auxiliary	Visits each train once.
Concurrent booking	O(S) + lock wait	O(1) per task	Seat search plus synchronized critical section.
Serialization	O(N)	O(N)	Writes all ticket objects and referenced objects.
Deserialization	O(N)	O(N)	Reconstructs the serialized ticket list.

4. Design / Proposed Solution

4.1 Object-Oriented Design & Encapsulation

The implementation uses an abstract Person class with Passenger as a subclass. Fields are private and accessed through methods, demonstrating data hiding. Train contains coaches and each Coach contains seats, while Ticket associates a passenger, train number, seat number and payment.

Method overriding is demonstrated through getRole() and toString(), while constructor overloading is demonstrated in Train. Runtime polymorphism is obtained when Passenger objects are handled through Person references. The service layer centralizes booking and cancellation rules.

4.2 Class Hierarchy and Architecture

4.2 CLASS HIERARCHY AND ARCHITECTURE

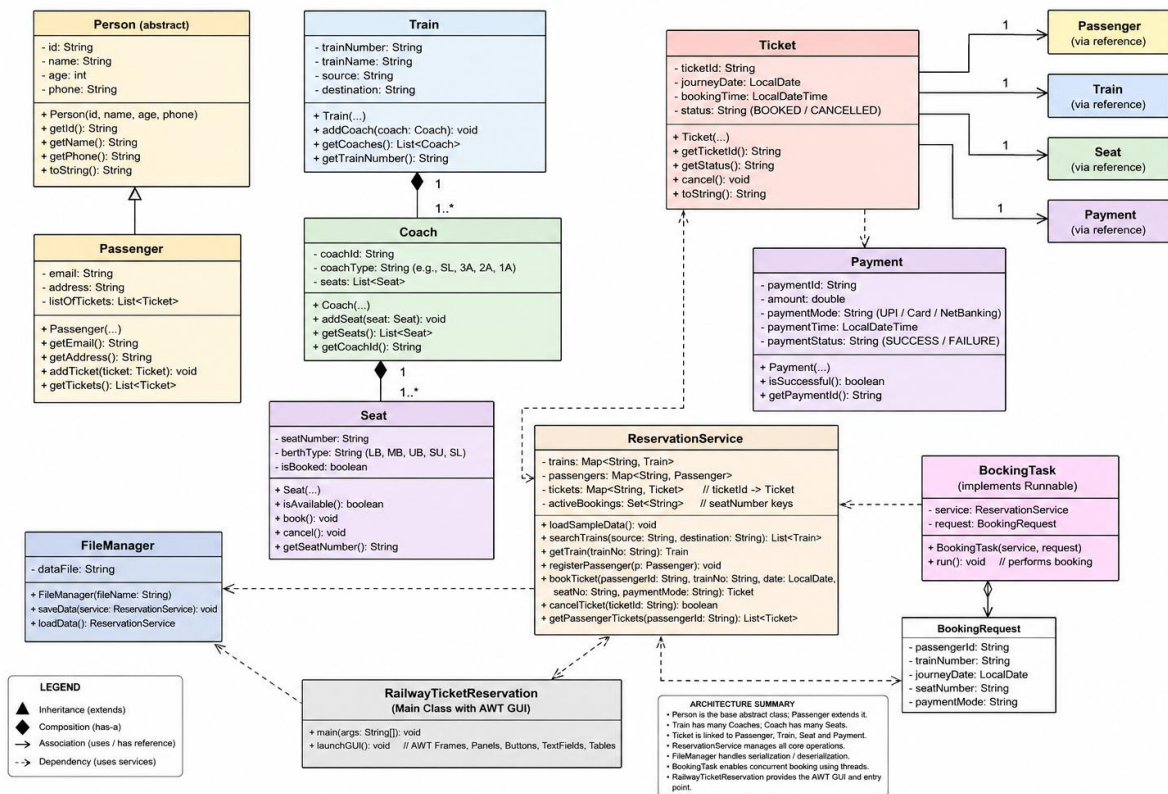


Figure 4.2: Class hierarchy and architecture

The diagram shows the inheritance relationship **Person** → **Passenger**, the **Train** → **Coach** → **Seat** composition, **Ticket** associations with **Passenger**, **Train**, **Seat** and **Payment**, and the supporting **ReservationService**, **FileManager**, **BookingTask** and **RailwayTicketReservation** classes.

4.3 System Architecture Overview

4.3 SYSTEM ARCHITECTURE OVERVIEW

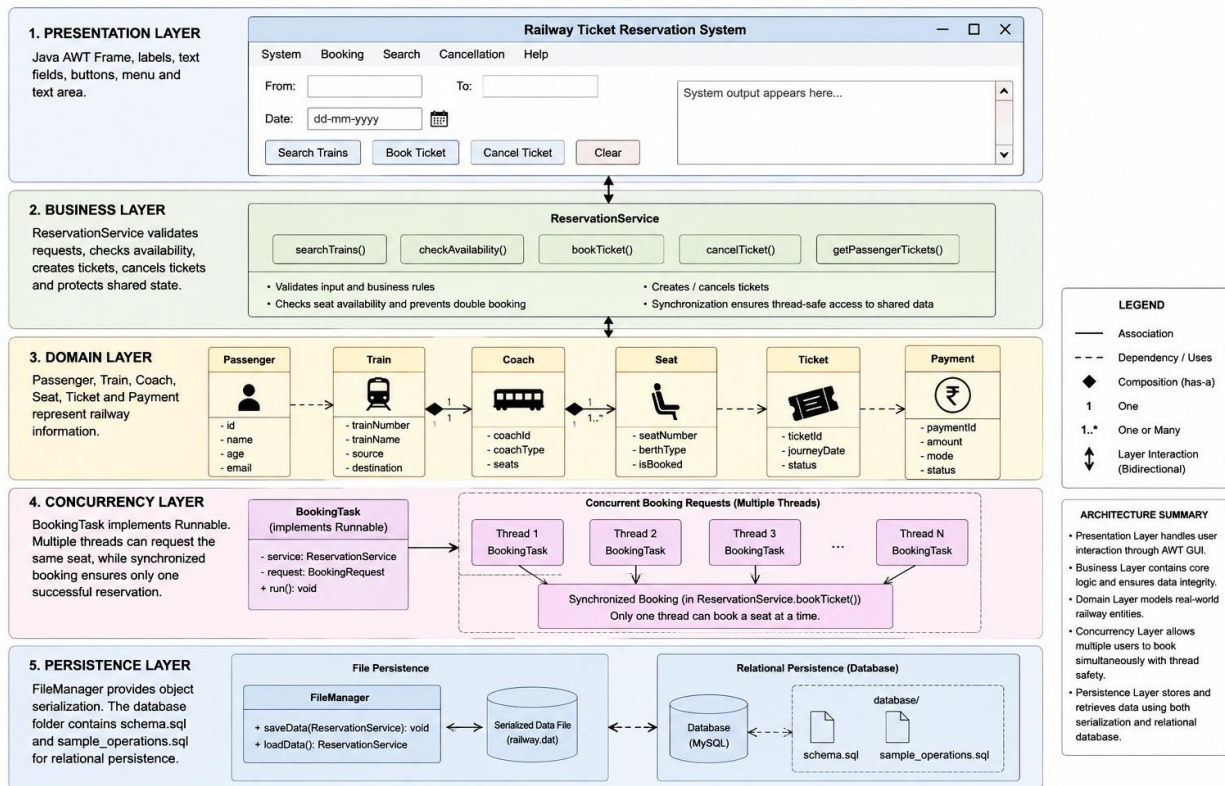


Figure 4.3: Five-layer system architecture overview

The presentation layer is implemented with Java AWT. ReservationService forms the business layer and performs validation, availability checks, booking and cancellation. The domain layer contains the railway entities, the concurrency layer contains BookingTask and synchronized booking, and the persistence layer contains FileManager serialization plus the supplied SQL files.

4.4 Multithreading and Synchronization Architecture

BookingTask implements Runnable and two threads are created in main() to request the same seat S1 on train 12601. ReservationService.bookTicket() is synchronized, and the activeBookings HashSet is updated inside the same critical section. Therefore, only one thread can successfully add the same train/seat booking key; the other receives SeatUnavailableException.

Thread priorities are demonstrated by assigning Thread.MAX_PRIORITY to Booking-Thread-1 and Thread.NORM_PRIORITY to Booking-Thread-2. Priority is only a scheduling hint; correctness comes from synchronization rather than priority ordering.

4.5 File and Database Persistence Architecture

FileManager writes the current List<Ticket> to tickets.dat using ObjectOutputStream and restores it using ObjectInputStream. The database folder separately provides schema.sql and sample_operations.sql to represent the relational persistence and CRUD part of the assignment; the current simplified Java source does not claim a live JDBC connection.

5. Algorithm / Pseudocode / Flowchart

5.1 Passenger Registration Algorithm

Core Working Principle: Passenger data is validated before the Passenger object is created and stored in the passengers HashMap.

5.1 Passenger Registration Algorithm

Algorithm RegisterPassenger(id, name, phone, age):

Input: passenger id, name, phone number and age

Output: validated Passenger stored in passengers HashMap

Step 1: Check that name contains at least two characters.

Step 2: Check that phone contains exactly ten digits.

Step 3: Check that age is between 1 and 120.

Step 4: If validation fails, throw InvalidPassengerException.

Step 5: Create Passenger object using validated values.

Step 6: Store passenger using passengers.put(id, passenger).

Step 7: Return successful registration.

Figure 5.1: Passenger registration algorithm

5.2 Train Search and Seat Availability Algorithm

Core Working Principle: The requested train is retrieved from the trains HashMap and every coach/seat is traversed to report the current status.

5.2 Train Search and Seat Availability Algorithm

Algorithm CheckAvailability(trainNumber):

Input: train number

Output: train details and status of every seat

Step 1: Find the train using trains.get(trainNumber).

Step 2: If train is null, report 'Train not found'.

Step 3: Traverse all coaches in the train.

Step 4: Traverse each coach's seats.

Step 5: Display each seat as AVAILABLE or BOOKED.

Step 6: Return the generated availability information.

Figure 5.2: Train search and seat availability algorithm

5.3 Concurrent Booking Algorithm

Core Working Principle: The synchronized bookTicket() method validates the request, atomically reserves a unique train/seat key, marks the seat booked, and creates the ticket.

5.3 Concurrent Booking Algorithm

```
Algorithm BookTicket(passengerId, trainNumber, seatNumber):
Input: passenger, train, seat and payment details
Output: confirmed Ticket or exception
Step 1: Enter the synchronized bookTicket() method.
Step 2: Validate passenger and train references.
Step 3: Locate the requested seat.
Step 4: Build bookingKey = trainNumber + '|' + seatNumber.
Step 5: Atomically add bookingKey to activeBookings HashSet.
Step 6: If key already exists, throw SeatUnavailableException.
Step 7: If seat status is BOOKED, remove key and throw exception.
Step 8: Mark seat as BOOKED.
Step 9: Create Payment and Ticket objects and store the ticket.
Step 10: Return the confirmed ticket.
```

Figure 5.3: Concurrent booking algorithm

5.4 Concurrent Booking Flowchart

5.4 Concurrent Booking Flow

```
[START]
|
[Receive booking request]
|
[Validate passenger and train]
|
Invalid --> [InvalidPassengerException / BookingException]
|
Valid
|
[Enter synchronized section]
|
[Check seat availability]
|
Booked --> [SeatUnavailableException]
|
Available
|
[Reserve seat + create ticket]
|
[Store ticket and booking key]
|
[Save / serialize if required]
|
[SUCCESS]
|
[END]
```

Figure 5.4: Concurrent booking flowchart

5.5 CONCURRENT BOOKING FLOWCHART

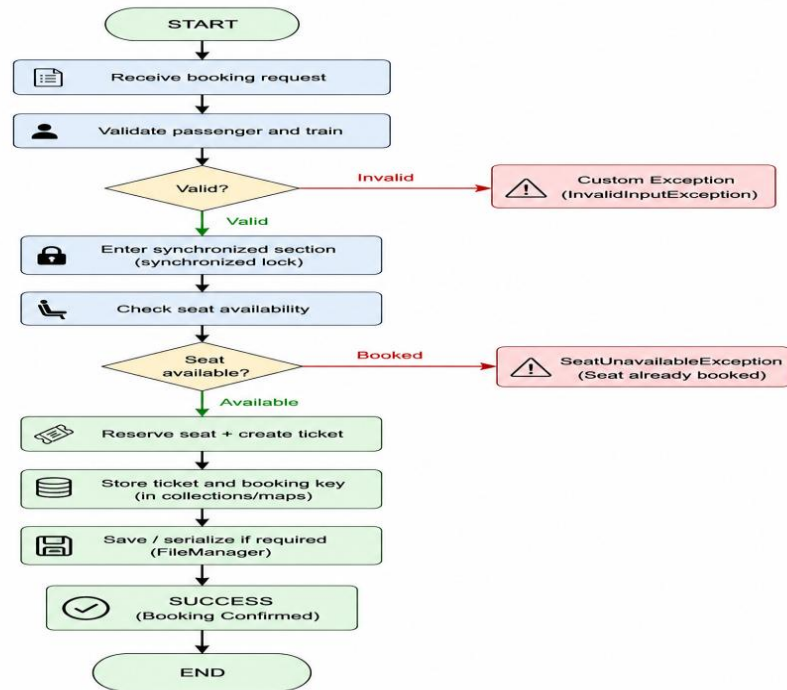


Figure 5.5: Graphical concurrent booking flowchart

5.5 Cancellation Algorithm

5.5 Cancellation Algorithm

Algorithm CancelTicket(ticketId):

Input: ticket id

Output: cancelled ticket and released seat

Step 1: Find the ticket using tickets.get(ticketId).

Step 2: If ticket does not exist, throw ReservationException.

Step 3: If ticket is already CANCELLED, throw ReservationException.

Step 4: Find the train and seat referenced by the ticket.

Step 5: Release the seat by changing status to AVAILABLE.

Step 6: Remove the train/seat booking key from activeBookings.

Step 7: Change ticket status to CANCELLED.

Step 8: Report successful cancellation.

Figure 5.6: Ticket cancellation algorithm

5.6 File Serialization / Deserialization Algorithm

5.6 Serialization / Deserialization Algorithm

Algorithm SaveAndRestoreTickets():

Input: current ticket collection

Output: ticket list restored from disk

Step 1: Open ObjectOutputStream for tickets.dat.

Step 2: Write the List<Ticket> using writeObject().

Step 3: Close the stream using try-with-resources.

Step 4: Open ObjectInputStream for tickets.dat.

Step 5: Read the serialized object using readObject().

Step 6: Cast the restored object to List<Ticket>.

Step 7: Return the restored ticket list.

Figure 5.7: Serialization and deserialization algorithm

6. Implementation / Source Code

File: RailwayTicketReservation.java | Language: Java | Target: Standard JVM

Compile and run in the project folder:

```
javac RailwayTicketReservation.java
java RailwayTicketReservation
```

The following pages reproduce the actual source file supplied with the simple project as code screenshots, matching the implementation used for the algorithms above.

RailwayTicketReservation.java — Source Code (Part 1)

```
1  import java.io.*;
2  import java.util.*;
3
4  /*
5   * CSA09 - Railway Ticket Reservation System
6   * Simple single-file implementation for easy VS Code execution.
7   *
8   * Demonstrates:
9   * OOP, inheritance, encapsulation, overloading/overriding,
10  * Collections, Generics, Iterator, exceptions, multithreading,
11  * synchronization, AWT GUI, file handling and serialization.
12  *
13  * JDBC/SQLite is kept in a separate optional file because the SQLite
14  * JDBC driver is an external dependency.
15  */
16
17  public class RailwayTicketReservation {
18
19      // ----- EXCEPTIONS -----
20
21      static class ReservationException extends Exception {
22          public ReservationException(String message) {
23              super(message);
24          }
25      }
26
27      static class InvalidPassengerException extends ReservationException {
28          public InvalidPassengerException(String message) {
29              super(message);
30          }
31      }
32
33      static class SeatUnavailableException extends ReservationException {
34          public SeatUnavailableException(String message) {
35              super(message);
36          }
37      }
38
39      static class BookingException extends ReservationException {
40          public BookingException(String message) {
41              super(message);
42          }
43      }
44  }
```

Figure 6.1: RailwayTicketReservation.java source code — Part 1

6. Implementation / Source Code (Continued - Part 2)

RailwayTicketReservation.java — Source Code (Part 2)

```
43     }
44
45     // ----- OOP: INHERITANCE -----
46
47     static abstract class Person implements Serializable {
48         private final String id;
49         private String name;
50         private String phone;
51
52         public Person(String id, String name, String phone) {
53             this.id = id;
54             this.name = name;
55             this.phone = phone;
56         }
57
58         public String getId() { return id; }
59         public String getName() { return name; }
60         public String getPhone() { return phone; }
61
62         public void setName(String name) { this.name = name; }
63         public void setPhone(String phone) { this.phone = phone; }
64
65         public abstract String getRole();
66
67         @Override
68         public String toString() {
69             return id + " - " + name + " - " + phone;
70         }
71     }
72
73     static class Passenger extends Person {
74         private int age;
75
76         public Passenger(String id, String name, String phone, int age)
77             throws InvalidPassengerException {
78             super(id, validateName(name), validatePhone(phone));
79             if (age < 1 || age > 120) {
80                 throw new InvalidPassengerException("Age must be between 1 and 120.");
81             }
82             this.age = age;
83         }
84     }
```

Figure 6.2: RailwayTicketReservation.java source code — Part 2

6. Implementation / Source Code (Continued - Part 3)

RailwayTicketReservation.java — Source Code (Part 3)

```
85     private static String validateName(String name)
86     throws InvalidPassengerException {
87         if (name == null || name.trim().length() < 2) {
88             throw new InvalidPassengerException("Passenger name is invalid.");
89         }
90         return name.trim();
91     }
92
93     private static String validatePhone(String phone)
94     throws InvalidPassengerException {
95         if (phone == null || !phone.matches("\\d{10}")) {
96             throw new InvalidPassengerException(
97                 "Phone number must contain exactly 10 digits.");
98         }
99         return phone;
100    }
101
102    public int getAge() { return age; }
103
104    @Override
105    public String getRole() {
106        return "Passenger";
107    }
108
109    @Override
110    public String toString() {
111        return "Passenger{" + super.toString() + ", age=" + age + "}";
112    }
113 }
114
115 enum SeatStatus {
116     AVAILABLE, BOOKED
117 }
118
119 static class Seat implements Serializable {
120     private final String seatNumber;
121     private SeatStatus status = SeatStatus.AVAILABLE;
122
123     public Seat(String seatNumber) {
124         this.seatNumber = seatNumber;
125     }
126 }
```

Figure 6.3: RailwayTicketReservation.java source code — Part 3

6. Implementation / Source Code (Continued - Part 4)

RailwayTicketReservation.java — Source Code (Part 4)

```
127     public String getSeatNumber() { return seatNumber; }
128     public SeatStatus getStatus() { return status; }
129
130     public void book() { status = SeatStatus.BOOKED; }
131     public void release() { status = SeatStatus.AVAILABLE; }
132
133     @Override
134     public String toString() {
135         return seatNumber + " [" + status + "];"
136     }
137 }
138
139 static class Coach implements Serializable {
140     private final String coachNumber;
141     private final ArrayList<Seat> seats = new ArrayList<>();
142
143     public Coach(String coachNumber, int seatCount) {
144         this.coachNumber = coachNumber;
145         for (int i = 1; i <= seatCount; i++) {
146             seats.add(new Seat("S" + i));
147         }
148     }
149
150     public String getCoachNumber() { return coachNumber; }
151     public ArrayList<Seat> getSeats() { return seats; }
152
153     public Seat findSeat(String seatNumber) {
154         for (Seat seat : seats) {
155             if (seat.getSeatNumber().equalsIgnoreCase(seatNumber)) {
156                 return seat;
157             }
158         }
159         return null;
160     }
161 }
162
163 static class Train implements Serializable {
164     private final String trainNumber;
165     private final String trainName;
166     private final String source;
167     private final String destination;
168     private final ArrayList<Coach> coaches = new ArrayList<>();
```

Figure 6.4: RailwayTicketReservation.java source code — Part 4

6. Implementation / Source Code (Continued - Part 5)

RailwayTicketReservation.java — Source Code (Part 5)

```
169
170     public Train(String trainNumber, String trainName,
171                 String source, String destination) {
172         this.trainNumber = trainNumber;
173         this.trainName = trainName;
174         this.source = source;
175         this.destination = destination;
176     }
177
178     // Method overloading
179     public Train(String trainNumber, String trainName) {
180         this(trainNumber, trainName, "Unknown", "Unknown");
181     }
182
183     public String getTrainNumber() { return trainNumber; }
184     public String getTrainName() { return trainName; }
185     public String getSource() { return source; }
186     public String getDestination() { return destination; }
187
188     public void addCoach(Coach coach) {
189         coaches.add(coach);
190     }
191
192     public ArrayList<Coach> getCoaches() { return coaches; }
193
194     public Seat findSeat(String seatNumber) {
195         for (Coach coach : coaches) {
196             Seat seat = coach.findSeat(seatNumber);
197             if (seat != null) return seat;
198         }
199         return null;
200     }
201
202     @Override
203     public String toString() {
204         return trainNumber + " - " + trainName + " : "
205             + source + " -> " + destination;
206     }
207 }
208
209 static class Payment implements Serializable {
210     private final String paymentId;
```

Figure 6.5: RailwayTicketReservation.java source code — Part 5

6. Implementation / Source Code (Continued - Part 6)

RailwayTicketReservation.java — Source Code (Part 6)

```
211     private final double amount;
212     private final String method;
213
214     public Payment(String paymentId, double amount, String method) {
215         this.paymentId = paymentId;
216         this.amount = amount;
217         this.method = method;
218     }
219
220     public String getPaymentId() { return paymentId; }
221     public double getAmount() { return amount; }
222     public String getMethod() { return method; }
223 }
224
225 enum TicketStatus {
226     CONFIRMED, CANCELLED
227 }
228
229 static class Ticket implements Serializable {
230     private final String ticketId;
231     private final Passenger passenger;
232     private final String trainNumber;
233     private final String seatNumber;
234     private final Payment payment;
235     private TicketStatus status;
236
237     public Ticket(String ticketId, Passenger passenger,
238                 String trainNumber, String seatNumber,
239                 Payment payment) {
240         this.ticketId = ticketId;
241         this.passenger = passenger;
242         this.trainNumber = trainNumber;
243         this.seatNumber = seatNumber;
244         this.payment = payment;
245         this.status = TicketStatus.CONFIRMED;
246     }
247
248     public String getTicketId() { return ticketId; }
249     public Passenger getPassenger() { return passenger; }
250     public String getTrainNumber() { return trainNumber; }
251     public String getSeatNumber() { return seatNumber; }
252     public TicketStatus getStatus() { return status; }
```

Figure 6.6: RailwayTicketReservation.java source code — Part 6

6. Implementation / Source Code (Continued - Part 7)

RailwayTicketReservation.java — Source Code (Part 7)

```
253     public Payment getPayment() { return payment; }
254
255     public void cancel() {
256         status = TicketStatus.CANCELLED;
257     }
258
259     @Override
260     public String toString() {
261         return "\nTicket ID : " + ticketId
262             + "\nPassenger : " + passenger.getName()
263             + "\nTrain      : " + trainNumber
264             + "\nSeat       : " + seatNumber
265             + "\nAmount     : Rs." + payment.getAmount()
266             + "\nPayment    : " + payment.getMethod()
267             + "\nStatus     : " + status;
268     }
269 }
270
271 // ----- SERVICE / COLLECTIONS -----
272
273 static class ReservationService {
274     // HashMap
275     private final HashMap<String, Train> trains = new HashMap<>();
276
277     // HashMap
278     private final HashMap<String, Passenger> passengers = new HashMap<>();
279
280     // HashMap
281     private final HashMap<String, Ticket> tickets = new HashMap<>();
282
283     // HashSet prevents duplicate active booking keys
284     private final HashSet<String> activeBookings = new HashSet<>();
285
286     private int ticketCounter = 1001;
287     private int paymentCounter = 501;
288
289     public void addTrain(Train train) {
290         trains.put(train.getTrainNumber(), train);
291     }
292
293     public void addPassenger(Passenger passenger) {
294         passengers.put(passenger.getId(), passenger);
```

Figure 6.7: RailwayTicketReservation.java source code — Part 7

6. Implementation / Source Code (Continued - Part 8)

RailwayTicketReservation.java — Source Code (Part 8)

```
295     }
296
297     public Train searchTrain(String trainNumber) {
298         return trains.get(trainNumber);
299     }
300
301     public Collection<Train> getAllTrains() {
302         return trains.values();
303     }
304
305     public Passenger getPassenger(String id) {
306         return passengers.get(id);
307     }
308
309     public synchronized Ticket bookTicket(
310         String passengerId,
311         String trainNumber,
312         String seatNumber,
313         double amount,
314         String paymentMethod)
315         throws ReservationException {
316
317         Passenger passenger = passengers.get(passengerId);
318         if (passenger == null) {
319             throw new BookingException("Passenger not found.");
320         }
321
322         Train train = trains.get(trainNumber);
323         if (train == null) {
324             throw new BookingException("Train not found.");
325         }
326
327         Seat seat = train.findSeat(seatNumber);
328         if (seat == null) {
329             throw new BookingException("Seat not found.");
330         }
331
332         String bookingKey = trainNumber + "|" + seat.getSeatNumber();
333
334         // Atomic check + add inside synchronized method.
335         if (!activeBookings.add(bookingKey)) {
336             throw new SeatUnavailableException(
```

Figure 6.8: RailwayTicketReservation.java source code — Part 8

6. Implementation / Source Code (Continued - Part 9)

RailwayTicketReservation.java — Source Code (Part 9)

```
337         "Duplicate booking prevented. Seat already reserved.");
338     }
339
340     if (seat.getStatus() == SeatStatus.BOOKED) {
341         activeBookings.remove(bookingKey);
342         throw new SeatUnavailableException("Seat is already booked.");
343     }
344
345     seat.book();
346
347     String ticketId = "T" + ticketCounter++;
348     String paymentId = "PAY" + paymentCounter++;
349
350     Payment payment = new Payment(paymentId, amount, paymentMethod);
351     Ticket ticket = new Ticket(
352         ticketId, passenger, trainNumber,
353         seat.getSeatNumber(), payment);
354
355     tickets.put(ticketId, ticket);
356
357     return ticket;
358 }
359
360 public synchronized void cancelTicket(String ticketId)
361     throws ReservationException {
362
363     Ticket ticket = tickets.get(ticketId);
364
365     if (ticket == null) {
366         throw new ReservationException("Ticket not found.");
367     }
368
369     if (ticket.getStatus() == TicketStatus.CANCELLED) {
370         throw new ReservationException("Ticket is already cancelled.");
371     }
372
373     Train train = trains.get(ticket.getTrainNumber());
374     Seat seat = train.findSeat(ticket.getSeatNumber());
375
376     if (seat != null) {
377         seat.release();
378     }
```

Figure 6.9: RailwayTicketReservation.java source code — Part 9

6. Implementation / Source Code (Continued - Part 10)

RailwayTicketReservation.java — Source Code (Part 10)

```
379
380         activeBookings.remove(
381             ticket.getTrainNumber() + "|" + ticket.getSeatNumber());
382
383         ticket.cancel();
384     }
385
386     public Ticket getTicket(String ticketId) {
387         return tickets.get(ticketId);
388     }
389
390     public List<Ticket> getTickets() {
391         return new ArrayList<>(tickets.values());
392     }
393
394     // Explicit Iterator demonstration
395     public String getTrainListUsingIterator() {
396         StringBuilder result = new StringBuilder();
397         Iterator<Train> iterator = trains.values().iterator();
398
399         while (iterator.hasNext()) {
400             result.append(iterator.next()).append("\n");
401         }
402
403         return result.toString();
404     }
405
406     public String getSeatAvailability(String trainNumber) {
407         Train train = trains.get(trainNumber);
408         if (train == null) return "Train not found.";
409
410         StringBuilder result = new StringBuilder();
411         for (Coach coach : train.getCoaches()) {
412             result.append(coach.getCoachNumber()).append(": ");
413             for (Seat seat : coach.getSeats()) {
414                 result.append(seat.getSeatNumber())
415                     .append("=")
416                     .append(seat.getStatus())
417                     .append(" ");
418             }
419             result.append("\n");
420         }
```

Figure 6.10: RailwayTicketReservation.java source code — Part 10

6. Implementation / Source Code (Continued - Part 11)

RailwayTicketReservation.java — Source Code (Part 11)

```
421         return result.toString();
422     }
423 }
424
425 // ----- FILE HANDLING / SERIALIZATION -----
426
427 static class FileManager {
428     private static final String FILE_NAME = "tickets.dat";
429
430     public static void saveTickets(List<Ticket> tickets)
431         throws IOException {
432         try (ObjectOutputStream out =
433             new ObjectOutputStream(
434                 new FileOutputStream(FILE_NAME))) {
435             out.writeObject(tickets);
436         }
437     }
438
439     @SuppressWarnings("unchecked")
440     public static List<Ticket> loadTickets()
441         throws IOException, ClassNotFoundException {
442         try (ObjectInputStream in =
443             new ObjectInputStream(
444                 new FileInputStream(FILE_NAME))) {
445             return (List<Ticket>) in.readObject();
446         }
447     }
448 }
449
450 // ----- MULTITHREADING -----
451
452 static class BookingTask implements Runnable {
453     private final ReservationService service;
454     private final String passengerId;
455     private final String trainNumber;
456     private final String seatNumber;
457
458     public BookingTask(ReservationService service,
459                       String passengerId,
460                       String trainNumber,
461                       String seatNumber) {
462         this.service = service;
```

Figure 6.11: RailwayTicketReservation.java source code — Part 11

6. Implementation / Source Code (Continued - Part 12)

RailwayTicketReservation.java — Source Code (Part 12)

```
463         this.passengerId = passengerId;
464         this.trainNumber = trainNumber;
465         this.seatNumber = seatNumber;
466     }
467
468     @Override
469     public void run() {
470         try {
471             Ticket ticket = service.bookTicket(
472                 passengerId, trainNumber, seatNumber,
473                 850.0, "UPI");
474
475             System.out.println(
476                 Thread.currentThread().getName()
477                 + " SUCCESS: " + ticket.getTicketId());
478         } catch (ReservationException e) {
479             System.out.println(
480                 Thread.currentThread().getName()
481                 + " REJECTED: " + e.getMessage());
482         }
483     }
484 }
485
486 // ----- AWT GUI -----
487
488 static class RailwayGUI extends java.awt.Frame
489     implements java.awt.event.ActionListener {
490
491     private final ReservationService service;
492
493     private final java.awt.TextField passengerField =
494         new java.awt.TextField();
495
496     private final java.awt.TextField nameField =
497         new java.awt.TextField();
498
499     private final java.awt.TextField phoneField =
500         new java.awt.TextField();
501
502     private final java.awt.TextField trainField =
503         new java.awt.TextField();
504 }
```

Figure 6.12: RailwayTicketReservation.java source code — Part 12

6. Implementation / Source Code (Continued - Part 13)

RailwayTicketReservation.java — Source Code (Part 13)

```
505     private final java.awt.TextField seatField =
506         new java.awt.TextField();
507
508     private final java.awt.TextField ticketField =
509         new java.awt.TextField();
510
511     private final java.awt.TextArea output =
512         new java.awt.TextArea();
513
514     public RailwayGUI(ReservationService service) {
515         this.service = service;
516
517         setTitle("Railway Ticket Reservation System");
518         setSize(800, 600);
519         setLayout(new java.awt.BorderLayout());
520
521         java.awt.Panel form = new java.awt.Panel(
522             new java.awt.GridLayout(6, 2, 8, 8));
523
524         form.add(new java.awt.Label("Passenger ID"));
525         form.add(passengerField);
526
527         form.add(new java.awt.Label("Passenger Name"));
528         form.add(nameField);
529
530         form.add(new java.awt.Label("Phone"));
531         form.add(phoneField);
532
533         form.add(new java.awt.Label("Train Number"));
534         form.add(trainField);
535
536         form.add(new java.awt.Label("Seat Number"));
537         form.add(seatField);
538
539         form.add(new java.awt.Label("Ticket ID"));
540         form.add(ticketField);
541
542         add(form, java.awt.BorderLayout.NORTH);
543
544         java.awt.Panel buttons = new java.awt.Panel(
545             new java.awt.FlowLayout());
546
```

Figure 6.13: RailwayTicketReservation.java source code — Part 13

6. Implementation / Source Code (Continued - Part 14)

RailwayTicketReservation.java — Source Code (Part 14)

```
547         String[] buttonNames = {
548             "Search Train",
549             "Availability",
550             "Book Ticket",
551             "Cancel Ticket",
552             "Ticket Details",
553             "Clear",
554             "Exit"
555         };
556
557         for (String name : buttonNames) {
558             java.awt.Button button =
559                 new java.awt.Button(name);
560             button.addActionListener(this);
561             buttons.add(button);
562         }
563
564         add(buttons, java.awt.BorderLayout.CENTER);
565
566         output.setEditable(false);
567         add(output, java.awt.BorderLayout.SOUTH);
568
569         // Menu
570         java.awt.MenuBar menuBar = new java.awt.MenuBar();
571         java.awt.Menu menu = new java.awt.Menu("System");
572
573         java.awt.MenuItem exit =
574             new java.awt.MenuItem("Exit");
575         exit.addActionListener(e -> dispose());
576
577         menu.add(exit);
578         menuBar.add(menu);
579         setMenuBar(menuBar);
580
581         addWindowListener(new java.awt.event.WindowAdapter() {
582             @Override
583             public void windowClosing(
584                 java.awt.event.WindowEvent e) {
585                 dispose();
586             }
587         });
588     }
```

Figure 6.14: RailwayTicketReservation.java source code — Part 14

6. Implementation / Source Code (Continued - Part 15)

RailwayTicketReservation.java — Source Code (Part 15)

```
589         setVisible(true);
590     }
591
592     @Override
593     public void actionPerformed(
594         java.awt.event.ActionEvent e) {
595
596         String command = e.getActionCommand();
597
598         try {
599             switch (command) {
600                 case "Search Train":
601                     Train train =
602                         service.searchTrain(
603                             trainField.getText().trim());
604
605                     output.setText(
606                         train == null
607                         ? "Train not found."
608                         : train.toString());
609                     break;
610
611                 case "Availability":
612                     output.setText(
613                         service.getSeatAvailability(
614                             trainField.getText().trim()));
615                     break;
616
617                 case "Book Ticket":
618                     // Create passenger if it doesn't exist.
619                     String pid =
620                         passengerField.getText().trim();
621
622                     Passenger passenger =
623                         service.getPassenger(pid);
624
625                     if (passenger == null) {
626                         passenger = new Passenger(
627                             pid,
628                             nameField.getText().trim(),
629                             phoneField.getText().trim(),
630                             20);
631                     }
632                 }
633             }
634         }
635     }
636 }
```

Figure 6.15: RailwayTicketReservation.java source code — Part 15

6. Implementation / Source Code (Continued - Part 16)

RailwayTicketReservation.java — Source Code (Part 16)

```
631         service.addPassenger(passenger);
632     }
633
634     Ticket ticket = service.bookTicket(
635         pid,
636         trainField.getText().trim(),
637         seatField.getText().trim(),
638         850.0,
639         "UPI");
640
641     ticketField.setText(ticket.getTicketId());
642     output.setText(ticket.toString());
643     break;
644
645     case "Cancel Ticket":
646         service.cancelTicket(
647             ticketField.getText().trim());
648         output.setText(
649             "Ticket cancelled successfully.");
650         break;
651
652     case "Ticket Details":
653         Ticket found =
654             service.getTicket(
655                 ticketField.getText().trim());
656
657         output.setText(
658             found == null
659                 ? "Ticket not found."
660                 : found.toString());
661         break;
662
663     case "Clear":
664         passengerField.setText("");
665         nameField.setText("");
666         phoneField.setText("");
667         trainField.setText("");
668         seatField.setText("");
669         ticketField.setText("");
670         output.setText("");
671         break;
672
```

Figure 6.16: RailwayTicketReservation.java source code — Part 16

6. Implementation / Source Code (Continued - Part 17)

RailwayTicketReservation.java — Source Code (Part 17)

```
673             case "Exit":
674                 dispose();
675                 break;
676
677             default:
678                 break;
679         }
680     } catch (Exception ex) {
681         output.setText("ERROR: " + ex.getMessage());
682     }
683 }
684 }
685
686 // ----- MAIN -----
687
688 public static void main(String[] args) {
689
690     System.out.println("=====");
691     System.out.println("    RAILWAY TICKET RESERVATION SYSTEM");
692     System.out.println("=====");
693
694     ReservationService service = new ReservationService();
695
696     try {
697         // Demo passengers
698         service.addPassenger(
699             new Passenger(
700                 "P1001",
701                 "Arun Kumar",
702                 "9876543210",
703                 21));
704
705         service.addPassenger(
706             new Passenger(
707                 "P1002",
708                 "Priya Sharma",
709                 "9876501234",
710                 22));
711
712         // Demo train
713         Train train = new Train(
714             "12601",
```

Figure 6.17: RailwayTicketReservation.java source code — Part 17

6. Implementation / Source Code (Continued - Part 18)

RailwayTicketReservation.java — Source Code (Part 18)

```
715         "Chennai Express",
716         "Chennai",
717         "Mumbai");
718
719     train.addCoach(new Coach("C1", 10));
720     train.addCoach(new Coach("C2", 10));
721
722     service.addTrain(train);
723
724     System.out.println("\nAVAILABLE TRAINS:");
725     System.out.println(
726         service.getTrainListUsingIterator());
727
728     System.out.println("SEAT AVAILABILITY:");
729     System.out.println(
730         service.getSeatAvailability("12601"));
731
732     // Multithreading demonstration:
733     // Two threads try to book the same seat.
734     System.out.println(
735         "CONCURRENT BOOKING TEST:");
736
737     Thread t1 = new Thread(
738         new BookingTask(
739             service,
740             "P1001",
741             "12601",
742             "S1"),
743         "Booking-Thread-1");
744
745     Thread t2 = new Thread(
746         new BookingTask(
747             service,
748             "P1002",
749             "12601",
750             "S1"),
751         "Booking-Thread-2");
752
753     t1.setPriority(Thread.MAX_PRIORITY);
754     t2.setPriority(Thread.NORM_PRIORITY);
755
756     t1.start();
```

Figure 6.18: RailwayTicketReservation.java source code — Part 18

6. Implementation / Source Code (Continued - Part 19)

RailwayTicketReservation.java — Source Code (Part 19)

```
757         t2.start();
758
759         t1.join();
760         t2.join();
761
762         // Serialization
763         FileManager.saveTickets(service.getTickets());
764         System.out.println(
765             "\nSerialization completed: tickets.dat created.");
766
767         List<Ticket> restored =
768             FileManager.loadTickets();
769
770         System.out.println(
771             "Deserialization completed. Tickets restored: "
772             + restored.size());
773
774         System.out.println(
775             "\nOpening AWT GUI...");
776
777         new RailwayGUI(service);
778
779     } catch (Exception e) {
780         System.out.println(
781             "APPLICATION ERROR: " + e.getMessage());
782     }
783 }
784 }
```

Figure 6.19: RailwayTicketReservation.java source code — Part 19

7. Test Cases and Expected Verification

The following test cases are derived directly from the implemented features. The result column is intentionally left for the user's actual execution evidence rather than claiming an unverified run.

TC-ID	Scenario	Input	Expected Result	Actual Result
TC-01	Valid passenger	P1001, Arun Kumar, 9876543210, 21	Passenger created successfully	
TC-02	Invalid passenger age	Age 0	InvalidPassengerException	
TC-03	Invalid phone	Phone with non-10-digit value	InvalidPassengerException	
TC-04	Train search	Train 12601	Train details displayed	
TC-05	Seat availability	Train 12601	Seat statuses displayed	
TC-06	Valid booking	P1001 / 12601 / S1	Ticket created and seat becomes BOOKED	
TC-07	Duplicate concurrent booking	Two threads / same S1	One succeeds; one is rejected	
TC-08	Cancellation	Valid ticket ID	Seat released and ticket becomes CANCELLED	
TC-09	Invalid cancellation	Unknown ticket ID	ReservationException	
TC-10	Serialization save	Current ticket list	tickets.dat created	
TC-11	Serialization load	tickets.dat	Tickets restored from file	
TC-12	AWT GUI	Book/Cancel/Search actions	Corresponding GUI result displayed	

8. Execution Screenshots / Output

Paste your own VS Code terminal and AWT GUI screenshots in the spaces below after running the program. These areas are intentionally blank so the final report contains your actual execution evidence.

Figure 8.1: Compilation and initial console output

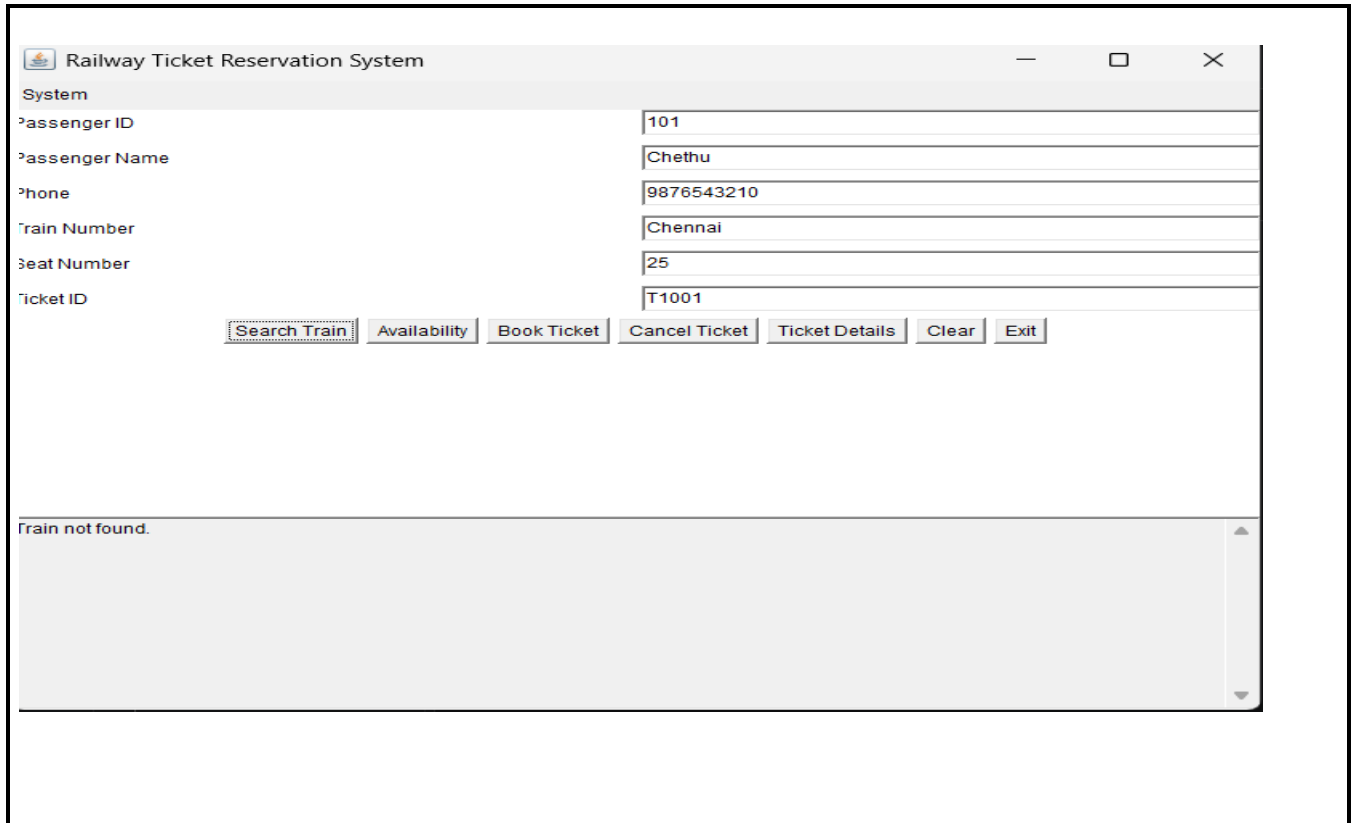


Figure 8.2: Concurrent booking result — one success and one rejection

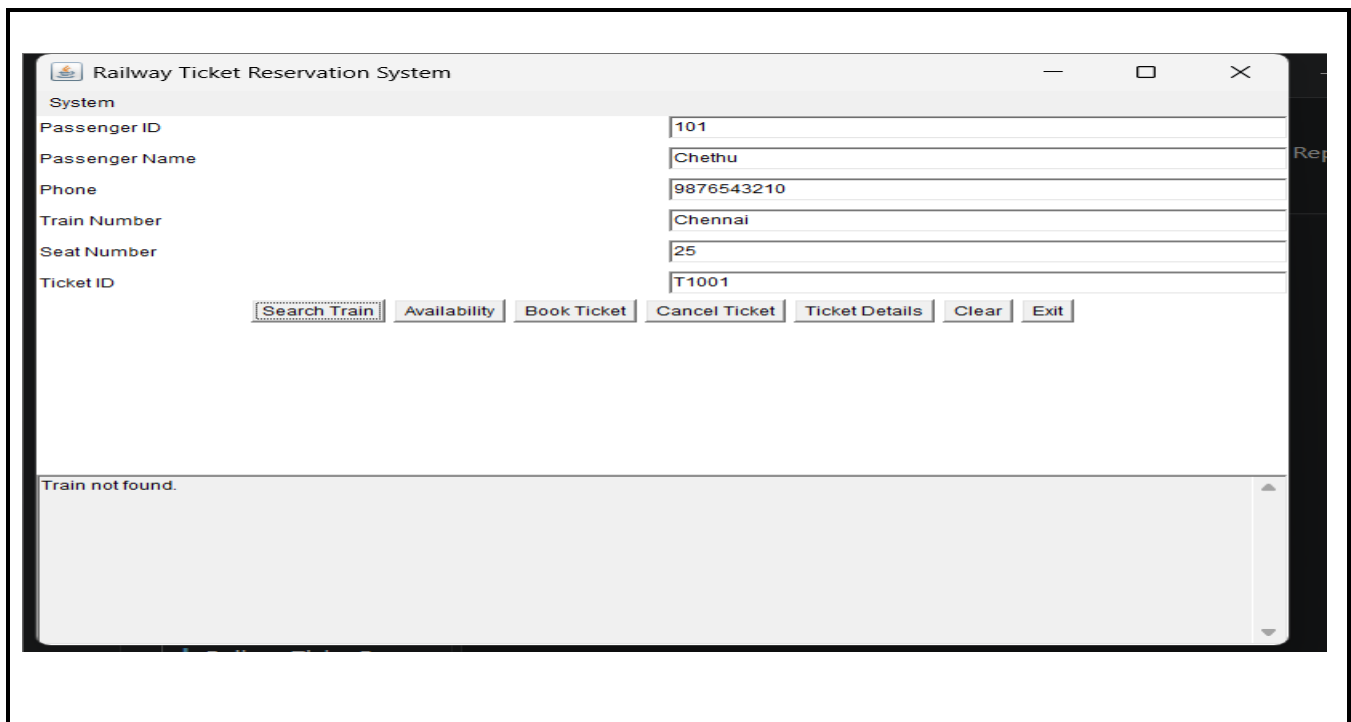


Figure 8.3: Serialization / deserialization output

Railway Ticket Reservation System

System

Passenger ID: 101

Passenger Name: Chethu

Phone: 9876543210

Train Number: 12601

Seat Number: S8

Ticket ID: T1002

Ticket ID : T1002
Passenger : Chethu
Train : 12601
Seat : S8
Amount : Rs.850.0
Payment : UPI
Status : CONFIRMED

Figure 8.4: AWT GUI — search and seat availability

Railway Ticket Reservation System

System

Passenger ID: 101

Passenger Name: Chethu

Phone: 9876543210

Train Number: 12601

Seat Number: S8

Ticket ID: T1002

C1: S1=BOOKED S2=AVAILABLE S3=AVAILABLE S4=AVAILABLE S5=AVAILABLE S6=AVAILABLE S7=AVAILABLE S8=BOOKED S9=AVA
C2: S1=AVAILABLE S2=AVAILABLE S3=AVAILABLE S4=AVAILABLE S5=AVAILABLE S6=AVAILABLE S7=AVAILABLE S8=AVAILABLE S9=

Figure 8.5: AWT GUI — successful ticket booking

The screenshot displays the 'Railway Ticket Reservation System' window. It features a form with the following fields and values:

System	
Passenger ID	101
Passenger Name	Chethu
Phone	9876543210
Train Number	12601
Seat Number	S8
Ticket ID	T1002

Below the form are several buttons: Search Train, Availability, **Book Ticket** (highlighted with a dashed border), Cancel Ticket, Ticket Details, Clear, and Exit.

A scrollable text area at the bottom displays the following ticket details:

```
Ticket ID : T1002
Passenger : Chethu
Train   : 12601
Seat    : S8
Amount  : Rs.850.0
Payment : UPI
Status  : CONFIRMED
```

Figure 8.6: AWT GUI — ticket cancellation / details

Railway Ticket Reservation System

System

Passenger ID

101

Passenger Name

Chethu

Phone

9876543210

Train Number

12601

Seat Number

S8

Ticket ID

T1002

Search Train

Availability

Book Ticket

Cancel Ticket

Ticket Details

Clear

Exit

Ticket cancelled successfully.

9. Analysis and Discussion

9.1 Collection Selection

HashMap is used for trains, passengers and tickets because the application frequently retrieves objects using unique identifiers. HashSet is used for activeBookings because a train/seat combination must be unique while active. ArrayList is suitable for coaches, seats and returned ticket lists because ordered traversal is simple and efficient.

9.2 Concurrency and Data Consistency

The main race condition occurs when two booking threads request the same seat. The synchronized bookTicket() method serializes access to the check-and-reserve sequence. The activeBookings HashSet performs the duplicate-key check within that same synchronized method, so the check and update are atomic with respect to other booking threads.

9.3 Exception Handling

Custom exceptions make failure cases explicit. InvalidPassengerException handles invalid name, phone and age; SeatUnavailableException handles duplicate or already-booked seats; BookingException handles missing passengers, trains or seats; ReservationException provides the common base type.

9.4 Thread Priority vs. Correctness

Thread priority is demonstrated in the main method but does not guarantee which booking thread executes first. The synchronized service method, rather than priority, is responsible for preventing duplicate seat reservation.

9.5 Serialization vs. Relational Persistence

Object serialization is convenient for saving and restoring the ticket object graph in one operation. Relational persistence is better suited to searchable, multi-record storage and structured CRUD. In this simplified implementation, SQL files are supplied for the relational design while the Java source directly implements serialization.

9.6 Maintainability and Scalability

Keeping the project in one Java file makes it easy for a student to compile and understand, but a production system should separate model, service, persistence, concurrency and GUI packages. JDBC integration, stronger input validation, database transactions and connection management can then be added without changing the domain model significantly.

10. Conclusion

The Railway Ticket Reservation System demonstrates the required Java concepts through a compact working design. The implementation combines OOP, collections, custom exceptions, synchronized concurrent booking, AWT event handling, serialization and SQL-based relational design. The most important concurrency safeguard is the synchronized booking operation, which ensures that a seat cannot be successfully reserved by two concurrent requests.

11. Individual Contribution

The project work represented by this report includes system design, Java implementation, algorithm preparation, source-code documentation, database SQL preparation, test-case design and report preparation. Replace this section with the exact student/group contribution details required by your submission format if necessary.

12. References

- Oracle. Java Platform, Standard Edition Documentation — Java Language and API concepts.
- Herbert Schildt. Java: The Complete Reference. McGraw-Hill.
- Bruce Eckel. Thinking in Java. Prentice Hall.
- Java documentation for AWT, Collections, Threads, synchronization and Object Serialization.
- Reference assignment report supplied for formatting and section organization.

13. One-Page Summary Write-Up

The Real-Time Railway Ticket Reservation System is a Java application designed to demonstrate object-oriented programming and concurrent reservation processing. Passenger, Train, Coach, Seat, Ticket and Payment classes model the railway domain, while ReservationService coordinates the core booking operations. An abstract Person class and Passenger subclass demonstrate inheritance and runtime polymorphism, and constructors/methods demonstrate overloading and overriding.

The application uses HashMap for identifier-based lookup, HashSet for duplicate active booking prevention, ArrayList for ordered collections, Generics for type safety and Iterator for explicit traversal. Custom exceptions handle invalid passenger data, unavailable seats and booking failures. BookingTask implements Runnable, and synchronized methods protect the shared reservation state when multiple threads attempt to book the same seat.

A Java AWT GUI provides search, availability, booking, cancellation, ticket details, clear and exit operations. FileManager demonstrates object serialization and deserialization using ObjectOutputStream and ObjectInputStream. The project also supplies relational SQL schema and CRUD operations for the database component. Actual execution screenshots are deliberately left as placeholders in Section 8 so the final report can contain the user's own VS Code and AWT output evidence.

4. Technology & Concurrency Comparative Summary

	Railway Ticket Reservation System	Traditional Database-Based System
Time & Space Complexity	$O(1)$ average HashMap/HashSet lookup; $O(N)$ storage	$O(\log N)$ indexed database lookup; persistent disk storage
Transaction Safety	Synchronized in-memory booking prevents duplicate seat allocation	ACID transactions provide persistent transaction safety
Deadlock Mitigation	Synchronized critical section and controlled shared-state access	Database lock manager and transaction timeouts
Operator Interface	Native Java AWT desktop GUI	Database/enterprise application interface
Recommended Role	Fast railway seat lookup, concurrent booking and local reservation processing	Persistent large-scale railway reservation database

5. Design Declaration, Project-Specific SDG Relevance & Reflection

- **Design Choice:** Using HashMap provides fast average-time lookup for passengers, trains and tickets, while HashSet helps prevent duplicate active booking keys. Synchronization ensures that concurrent threads cannot successfully reserve the same seat.
- **SDG 8 (Decent Work & Economic Growth):** The Railway Ticket Reservation System improves the efficiency of railway reservation operations through automated passenger management, seat availability checking, ticket booking and cancellation.
- **SDG 9 (Industry, Innovation & Infrastructure):** The project demonstrates digital railway infrastructure using Java OOP, Collections, AWT GUI, multithreading, synchronization, serialization and database-oriented SQL design.
- **SDG 16 (Peace, Justice & Strong Institutions):** The system emphasizes reliable reservation processing, controlled concurrent operations, consistent ticket records and exception handling to reduce booking errors.
- **Key Learning:** Concurrent railway reservation requires careful synchronization and shared-state management. The project demonstrates how OOP, collections, exception handling and multithreading can be combined to build a reliable reservation system.